

Spoleczna Akademia Nauk

Instytut Technologii Informatycznych

Kierunek studiów:

INFORMATYKA

Przedmiot:

Systemy Szkieletowe

GearFlow – System rezerwacji sprzętu

Sprawozdanie z projektu

Yevhenii Kolisnyk

Nr albumu: 92385

Grupa: 3

Prowadzący:
mgr inż. Krystian Gumiński

Łódź 2026

Spis treści

1. Wstęp	2
2. Cel projektu	3
3. Przegląd istniejących rozwiązań.....	3
3.1. EZOfficeInventory.....	4
3.2. Asset Panda	5
3.3. Snipe-IT.....	5
4. Opis architektury.....	6
5. Opis implementacji	8
5.1. Frontend.....	8
5.2. Backend	10
5.3. Baza danych	13
5.4. Docker.....	14
5.5. Testy	14
5.6. Monitoring i powiadomienia.....	15
6. Podsumowanie i wnioski.....	16
Spis tabel	18
Spis rysunków.....	19
Bibliografia.....	20

1. Wstęp

Niniejsze sprawozdanie dokumentuje projekt GearFlow – aplikację webową służącą do zarządzania rezerwacjami sprzętu w organizacji. Projekt realizowany jest w ramach przedmiotu Systemy Szkieletowe na kierunku Informatyka Społecznej Akademii Nauk w Łodzi. Celem sprawozdania jest szczegółowe opisanie architektury, zastosowanych technologii oraz kluczowych decyzji projektowych podjętych w trakcie tworzenia systemu.

GearFlow rozwiązuje problem ręcznego śledzenia wypożyczeń sprzętu w firmach, uczelniach i innych organizacjach, gdzie pracownicy muszą rezerwować i zwracać urządzenia

takie jak laptopy, kamery, projektory czy narzędzia. System umożliwia zarówno pracownikom, jak i administratorom sprawne zarządzanie całym cyklem życia rezerwacji.

W dalszej części sprawozdania opisano cel projektu, przeprowadzono analizę istniejących rozwiązań rynkowych, szczegółowo omówiono architekturę systemu oraz poszczególne warstwy implementacji. Sprawozdanie zawiera również opis zastosowanego konteneryzowania przy użyciu Dockera, strategii testowania oraz mechanizmów monitorowania.

2. Cel projektu

Głównym celem projektu jest stworzenie kompletnej aplikacji webowej, która pozwoli pracownikom organizacji na samodzielne rezerwowanie sprzętu oraz umożliwi administratorom sprawną kontrolę nad stanami magazynowymi. System ma zastąpić tradycyjne, ręczne metody ewidencjonowania sprzętu – arkusze kalkulacyjne oraz zeszyty wypożyczeń.

Wymagania funkcjonalne systemu obejmują: rejestrację i logowanie użytkowników (w tym logowanie przez konto Microsoft Azure AD z domeną uczelnianą @student.san.edu.pl), przeglądanie listy dostępnego sprzętu z możliwością filtrowania i wyszukiwania, tworzenie i anulowanie rezerwacji, raportowanie defektów sprzętu po zakończeniu wypożyczenia, zarządzanie lokalizacjami i typami sprzętu przez administratora, podgląd statystyk w formie wykresów oraz eksport danych do formatu CSV i PDF.

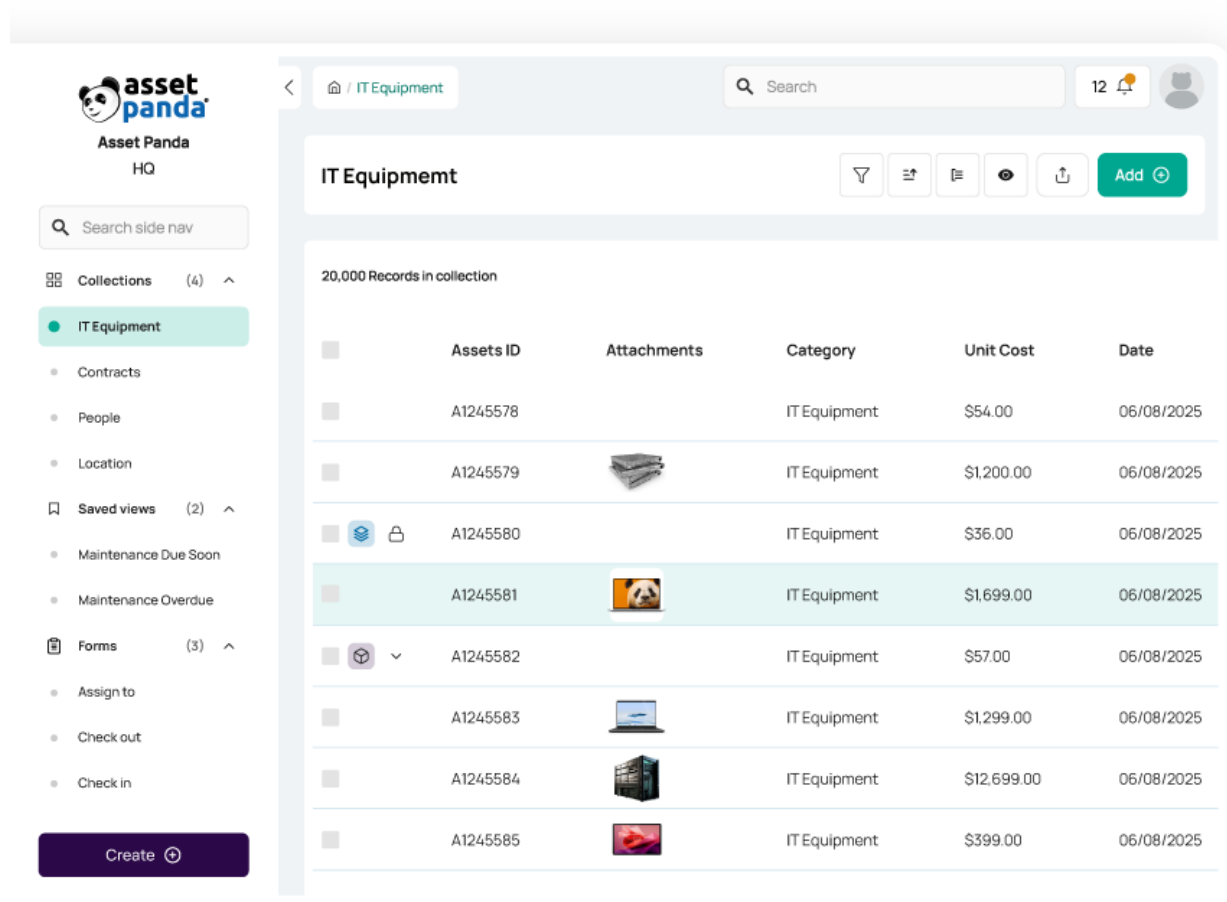
Wymagania нефunkcjonalne systemu to: bezpieczeństwo oparte na tokenach JWT przechowywanych w ciasteczkach HttpOnly, responsywny interfejs użytkownika zbudowany z komponentów biblioteki PrimeNG, skalowalność dzięki architekturze kontenerowej opartej na Dockerze, a także automatyczne powiadomienia e-mail przypominające o zbliżającym się terminie zwrotu sprzętu.

System wyróżnia dwa poziomy dostępu: pracownik (Employee), który może przeglądać sprzęt, tworzyć rezerwacje i zarządzać swoimi wypożyczeniami, oraz administrator (Admin), który dodatkowo może zarządzać katalogiem sprzętu, lokalizacjami, typami sprzętu, potwierdzać zwroty oraz przeglądać statystyki systemowe.

3. Przegląd istniejących rozwiązań

Na rynku dostępnych jest kilka systemów do zarządzania zasobami i rezerwacjami sprzętu. W niniejszym rozdziale dokonano analizy trzech wybranych rozwiązań, które stanowiły

punkt odniesienia podczas projektowania systemu GearFlow. Każde z analizowanych rozwiązań posiada zarówno zalety, jak i ograniczenia, które wpłynęły na kształt podejmowanych decyzji projektowych.



Rysunek 1. Przykładowy interfejs jednego z analizowanych systemów

3.1. EZOfficeInventory

EZOfficeInventory to zaawansowana platforma SaaS do zarządzania aktywami i rezerwacjami sprzętu, dostępna w modelu subskrypcyjnym. System oferuje szerokie możliwości śledzenia zasobów, zarządzania cyklem życia urządzeń, generowania raportów oraz integracji z zewnętrznymi systemami poprzez API REST. Platforma wspiera kody QR i kody kreskowe do identyfikacji sprzętu, a także zaawansowane przepływy zatwierdzania rezerwacji.

Główną wadą EZOfficeInventory z perspektywy małych organizacji jest model cenowy oparty na subskrypcji miesięcznej, który staje się kosztowny przy większej liczbie użytkowników. System nie oferuje możliwości wdrożenia on-premise, co może być

ograniczeniem dla organizacji z restrykcjami dotyczącymi przechowywania danych. Interfejs, choć rozbudowany, bywa oceniany jako nieczytelny dla nowych użytkowników.

3.2. Asset Panda

Asset Panda to platforma do zarządzania majątkiem firmy z funkcjami rezerwacji, serwisowania oraz śledzenia lokalizacji sprzętu. System wyróżnia się aplikacją mobilną na iOS i Android, możliwością tworzenia niestandardowych formularzy i przepływów pracy oraz integracją z popularnymi narzędziami business intelligence. Szczególną zaletą jest intuicyjny kreator raportów bez potrzeby znajomości SQL.

Podobnie jak EZOfficeInventory, Asset Panda funkcjonuje wyłącznie jako usługa chmurowa, co wiąże się z uzależnieniem od zewnętrznego dostawcy. Model licencjonowania jest nieprzejrysty – ceny dostępne jedynie po kontakcie z działem sprzedaży. System jest przystosowany głównie do dużych przedsiębiorstw i może być zbyt rozbudowany dla potrzeb uczelni lub małych zespołów badawczych.

3.3. Snipe-IT

Snipe-IT to otwarte oprogramowanie (open-source) do zarządzania aktywami IT, dostępne na licencji AGPL. W przeciwieństwie do poprzednich rozwiązań, może być wdrożone samodzielnie na własnym serwerze, co eliminuje koszty subskrypcji i zapewnia pełną kontrolę nad danymi. System oferuje zarządzanie zasobami, komponentami, akcesoriami oraz licencjami, a także API REST umożliwiające integrację z innymi narzędziami.

Ograniczeniem Snipe-IT jest skupienie się wyłącznie na zarządzaniu aktywami IT, bez dedykowanego modułu rezerwacji sprzętu z kalendarzem dostępności. Interfejs użytkownika, mimo funkcjonalności, jest przestarzały wizualnie i nie oferuje nowoczesnego doświadczenia porównywalnego z rozwiązaniami SaaS. Brak natywnej integracji z systemami SSO uczelni, takich jak Azure Active Directory, stanowi dodatkową barierę wdrożeniową.

Na tle analizowanych rozwiązań, GearFlow wyróżnia się: integracją z Microsoft Azure AD umożliwiającą logowanie kontem uczelnianym, dedykowanym przepływem zatwierdzania zwrotów sprzętu przez administratora, nowoczesnym interfejsem opartym na Angular i PrimeNG oraz możliwością pełnego wdrożenia lokalnego przy użyciu Docker Compose.

4. Opis architektury

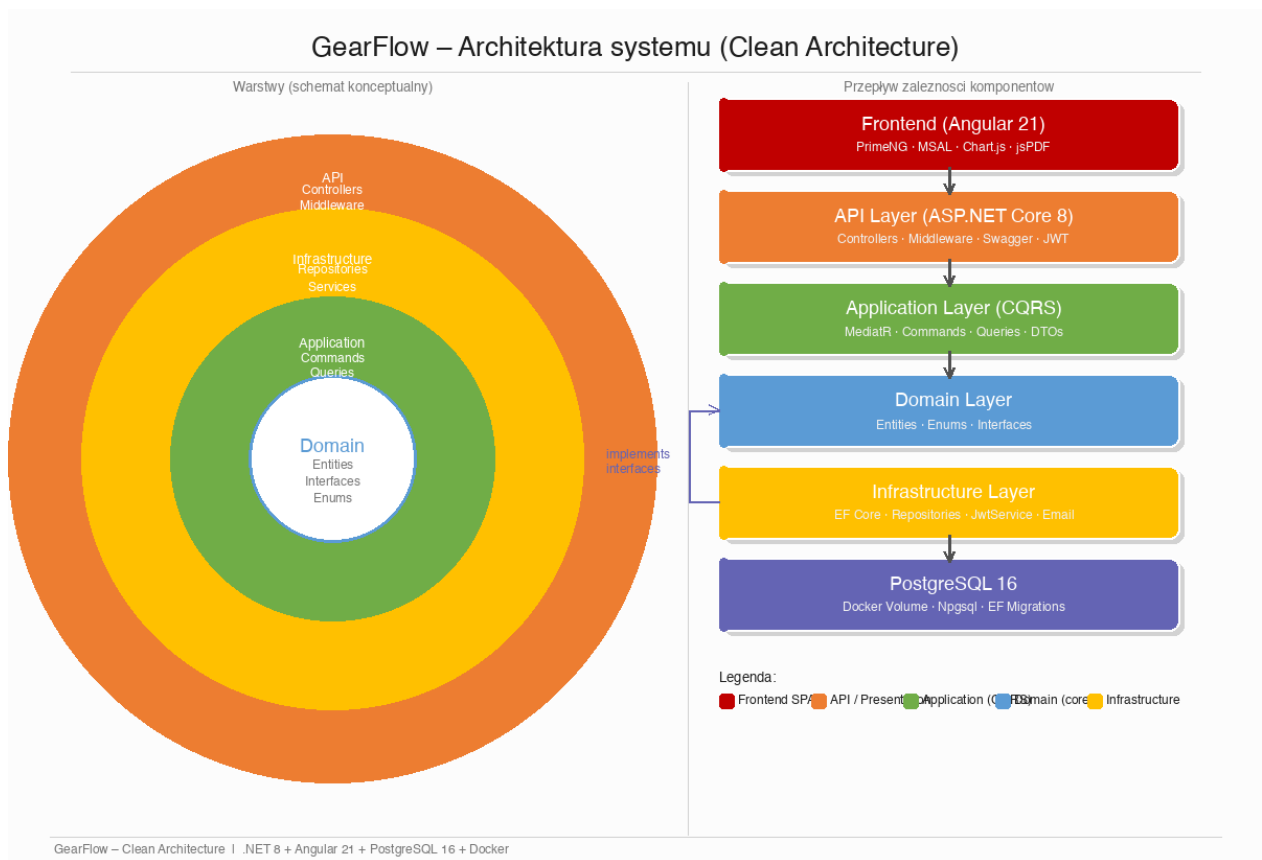
GearFlow zbudowany jest w oparciu o architekturę Clean Architecture zaproponowaną przez Roberta C. Martina. Architektura ta zakłada podział systemu na koncentryczne warstwy, przy czym wewnętrzne warstwy nie zależą od zewnętrznych. Podejście to zapewnia testowalność logiki biznesowej niezależnie od infrastruktury i frameworków.

Projekt backendowy składa się z czterech warstw: Domain, Application, Infrastructure oraz API. Warstwa Domain zawiera encje biznesowe i interfejsy repozytoriów, warstwa Application implementuje logikę aplikacji z użyciem wzorca CQRS (Command Query Responsibility Segregation) i biblioteki MediatR, warstwa Infrastructure dostarcza konkretne implementacje repozytoriów i serwisów, natomiast warstwa API eksponuje endpointy REST i obsługuje żądania HTTP.

Komunikacja między frontendem a backendem odbywa się za pośrednictwem REST API. Uwierzytelnianie bazuje na tokenach JWT przechowywanych w ciasteczkach HttpOnly z flagą Secure i SameSite=Strict, co chroni przed atakami CSRF i XSS. Tabela poniżej przedstawia główne komponenty architektury systemu.

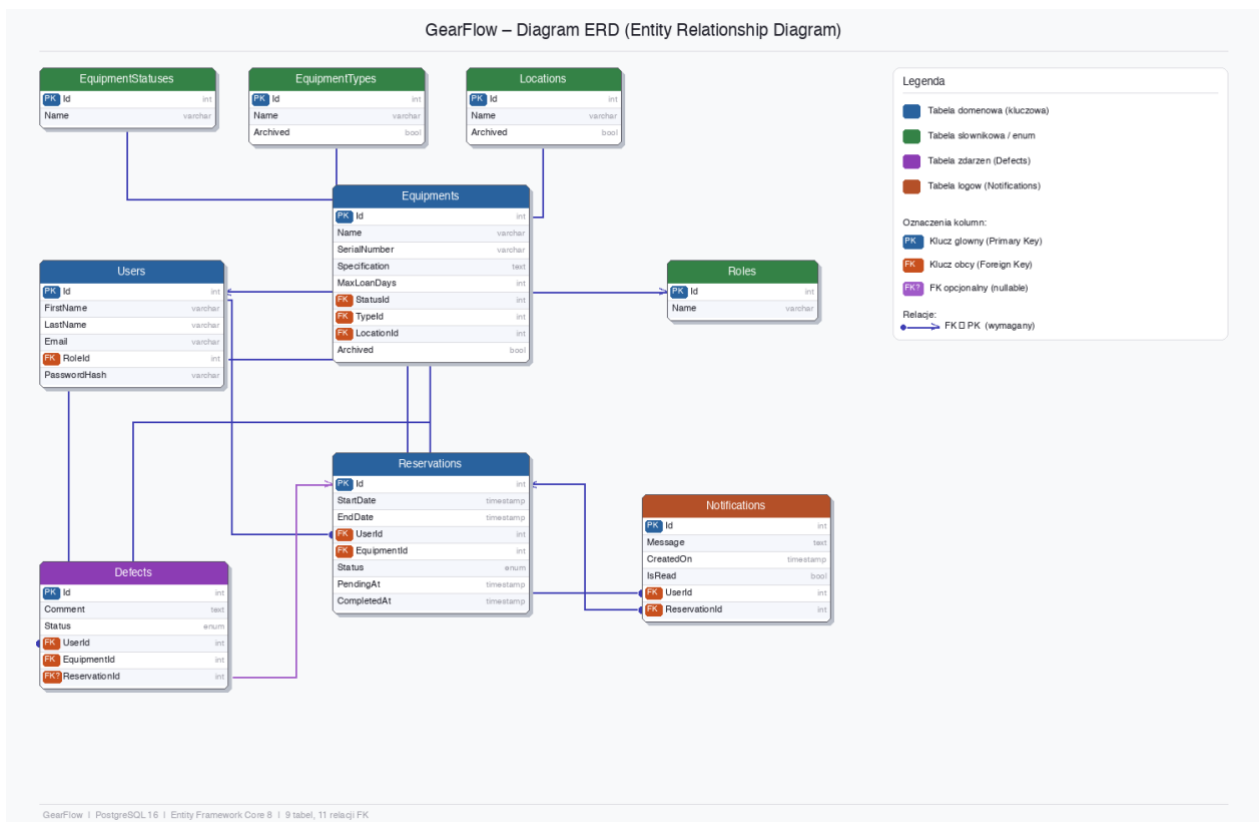
Tabela 1. Główne komponenty architektury systemu GearFlow

Lp.	Komponent	Technologia / Framework	Warstwa
1	Warstwa prezentacji (frontend)	Angular 21, PrimeNG 21, TypeScript	Prezentacja
2	Reverse proxy / serwer HTTP	Nginx (Alpine)	Prezentacja
3	REST API	ASP.NET Core 8, kontrolery MVC	API
4	Logika aplikacji (CQRS)	MediatR, Command/Query handlers	Aplikacja
5	Encje domenowe	C# POCO, EF Core annotations	Domena
6	Repozytorium / ORM	Entity Framework Core 8, Npgsql	Infrastruktura
7	Baza danych	PostgreSQL 16	Infrastruktura
8	Serwis JWT	Microsoft.IdentityModel.Tokens	Infrastruktura
9	Serwis e-mail	MailKit / MimeKit (SMTP)	Infrastruktura
10	Aktualizator statusów (background)	IHostedService / BackgroundService	Infrastruktura
11	Uwierzytelnianie Microsoft SSO	MSAL Angular, Azure Active Directory	Cross-cutting
12	Konteneryzacja	Docker, Docker Compose	DevOps



Rysunek 2. Diagram architektury warstwowej systemu GearFlow

Model danych systemu opiera się na dziewięciu tabelach bazodanowych: Users, Roles, Locations, Equipments, EquipmentTypes, EquipmentStatuses, Reservations, Defects oraz Notifications. Każda encja dziedziczy po klasie BaseEntity zawierającej identyfikator Id. Sprzęt może mieć jeden z pięciu statusów: Available (dostępny), Reserved (zarezerwowany), Borrowed (wypożyczony), Serviced (w serwisie) oraz Destroyed (zniszczony). Rezerwacje przechodzą przez cykl życia: Active → Pending (po zgłoszeniu zwrotu przez pracownika) → Completed lub Cancelled.



Rysunek 3. Diagram encji i relacji (ERD) bazy danych GearFlow

5. Opis implementacji

W niniejszym rozdziale opisano kluczowe aspekty implementacyjne poszczególnych warstw systemu GearFlow. Omówiono zastosowane technologie, strukturę kodu, istotne decyzje projektowe oraz mechanizmy bezpieczeństwa.

5.1. Frontend

Frontend aplikacji zbudowany jest w oparciu o framework Angular w wersji 21, z wykorzystaniem architektury komponentów standalone (bez modułów NgModule). Do budowania interfejsu użytkownika zastosowano bibliotekę PrimeNG 21, opartą na motywie Aura, która dostarcza gotowe, dostępne komponenty UI: tabele z paginacją, formularze, okna dialogowe, wykresy i menu nawigacji.

Zarządzanie stanem aplikacji opiera się na sygnałach (Signals) wprowadzonych w Angular 16+, co zastępuje tradycyjne podejście oparte na RxJS Subjects w przypadku lokalnego stanu komponentów. Do komunikacji HTTP wykorzystano HttpClient z interceptorami:

AuthInterceptor dołącza ciasteczkowe tokeny JWT do każdego żądania, a ErrorInterceptor centralnie obsługuje błędy serwera.

Routing aplikacji implementuje lazy loading – każdy widok ładowany jest dynamicznie na żądanie, co skraca czas ładowania aplikacji. Dostęp do tras chroniony jest przez trzy guardy: authGuard (wymagane zalogowanie), adminGuard (wymagana rola Admin) oraz guestGuard (przekierowanie zalogowanych użytkowników z formularzy logowania).

Aplikacja zawiera następujące widoki: lista sprzętu (z filtrowaniem po typie, lokalizacji, statusie i maksymalnej liczbie dni wypożyczenia), szczegóły sprzętu z formularzem rezerwacji i kalendarzem zajętych dat, lista rezerwacji użytkownika (aktywne, oczekujące, historia), panel administratora z oczekującymi zwrotami, zarządzanie lokalizacjami, zarządzanie typami sprzętu oraz panel statystyk dostępny wyłącznie dla administratora.

Panel statystyk wyświetla dane w formie wykresów: słupkowego (liczba rezerwacji w poszczególnych miesiącach roku bieżącego) i kołowego (rozkład rezerwacji według typów sprzętu), opartych na bibliotece Chart.js. Dane mogą być eksportowane do pliku PDF przy użyciu bibliotek jsPDF i jspdf-autotable.

EquipmentReservations

Yevhenii Koliisnyk

Equipments

Search

CSV

PDF

Name	Serial number	Specification	Max loan days	Location	Status	Type	
Dell XPS 15	LPT-001-2024	Intel Core i7-13700H, 32GB RAM, 1TB SSD, NVIDIA RTX 4060, 15.6" OLED	14	Warsaw HQ	Reserved	Laptop	
Lenovo ThinkPad X1 Carbon	LPT-002-2024	Intel Core i7-1365U, 16GB RAM, 512GB SSD, 14" IPS display, 1.12kg	14	Kraków Branch	Available	Laptop	
MacBook Pro 14"	LPT-003-2024	Apple M3 Pro, 18GB RAM, 512GB SSD, Liquid Retina XDR, 18h battery	7	Warsaw HQ	Reserved	Laptop	
HP EliteBook 840 G10	LPT-004-2024	Intel Core i5-1345U, 16GB RAM, 256GB SSD, 14" FHD, vPro security	14	Gdańsk Office	Available	Laptop	
ASUS ProArt Studiobook 16	LPT-005-2023	AMD Ryzen 9 7945HX, 64GB RAM, 2TB SSD, RTX 4070, OLED touchscreen	7	Wrocław Office	Reserved	Laptop	
Microsoft Surface Pro 9	LPT-006-2023	Intel Core i7-1265U, 16GB RAM, 256GB SSD, 13" PixelSense 2-in-1	10	Warsaw HQ	Available	Laptop	
Sony Alpha A7 IV	CAM-001-2024	33MP Full-Frame, 4K 60fps, 5-axis IBIS, dual CFexpress/SD slots	7	Gdańsk Office	Reserved	Camera	
Canon EOS R5	CAM-002-2024	45MP Full-Frame, 8K RAW video, IBIS, dual card slots, weather-sealed	7	Warsaw HQ	Available	Camera	
DJI Pocket 3	CAM-003-2023	4K/120fps, 3-axis gimbal, 1" CMOS sensor, built-in mic array	5	Storage Room	Available	Camera	
Nikon Z6 III	CAM-004-2024	24.5MP partial-stacked CMOS, 6K RAW, 120fps FHD, IBIS, dual card	7	Kraków Branch	Reserved	Camera	
GoPro Hero 12 Black	CAM-005-2023	5.3K/60fps, HyperSmooth 6.0, waterproof 10m, HDR, Enduro battery	5	Storage Room	Available	Camera	
DJI Air 3 Drone	CAM-006-2024	4K/60fps dual camera, 46min flight time, 20km range, obstacle avoidance	3	Storage Room	Available	Camera	
Epson EB-L200F	PRJ-001-2023	Full HD 1080p, 4500 lumens, laser, wireless LAN, portrait projection	3	Warsaw HQ	Available	Projector	
BenQ MH560	PRJ-002-2022	Full HD 1080p, 3800 lumens, SmartEco technology, 10W speaker	3	Storage Room	Serviced	Projector	
Optoma UHD38s	PRJ-003-2024	4K UHD, 4000 lumens, 240Hz gaming mode, HDR10, eARC HDMI	3	Kraków Branch	Available	Projector	
ViewSonic M2e	PRJ-004-2023	Full HD portable, 1000 lumens, Harman Kardon speakers, Wi-Fi/BT, 3h battery	5	Wrocław Office	Reserved	Projector	
Bosch GSR 18V-55	PTL-001-2023	18V cordless drill/driver, 55Nm torque, 2x2.0Ah batteries, charger	5	Storage Room	Available	Power Tool	

Rysunek 4. Widok listy sprzętu z filtrowaniem



Rysunek 5. Panel statystyk administratora

5.2. Backend

Backend systemu oparty jest na platformie ASP.NET Core 8 w języku C#. Zastosowanie architektury Clean Architecture w połączeniu ze wzorcem CQRS i biblioteką MediatR zapewnia wyraźny podział odpowiedzialności: komendy (Commands) modyfikują stan systemu, a zapytania (Queries) jedynie odczytują dane.

Uwierzytelnianie opiera się na dwóch mechanizmach. Standardowe logowanie email/hasło używa algorytmu BCrypt do hashowania haseł i generuje token JWT przechowywany w ciasteczku HttpOnly z 7-dniowym czasem ważności. Alternatywnie, użytkownicy mogą zalogować się kontem Microsoft – system weryfikuje token MSAL po stronie serwera i akceptuje wyłącznie adresy z domeny @student.san.edu.pl, co ogranicza dostęp do kont uczelnianych.

Autoryzacja zaimplementowana jest przez politykę AdminOnly, weryfikującą claim ClaimTypes.Role o wartości 1 (Admin). Endpointy tworzenia/modyfikacji/archiwizacji sprzętu, lokalizacji i typów sprzętu, potwierdzania zwrotów oraz dostępu do statystyk objęte są tą polityką.

Obsługa błędów scentralizowana jest w middleware ExceptionHandlingMiddleware, który przechwytuje nieobsłużone wyjątki i mapuje je na odpowiednie kody HTTP:

KeyNotFoundException → 404 Not Found, UnauthorizedAccessException → 401 Unauthorized, ArgumentException → 400 Bad Request, pozostałe wyjątki → 500 Internal Server Error.

API posiada endpointy health check pod ścieżkami /health (prosty status tekstowy) oraz /health/detail (szczegółowy JSON z informacją o dostępności bazy danych). Swagger UI dostępny jest w środowisku deweloperskim pod adresem /swagger.

Tabela 2. Główne endpointy REST API systemu GearFlow

Metoda	Ścieżka	Opis	Autoryzacja
POST	/api/auth/register	Rejestracja nowego użytkownika	Publiczny
POST	/api/auth/login	Logowanie (email + hasło)	Publiczny
POST	/api/auth/microsoft	Logowanie kontem Microsoft SSO	Publiczny
GET	/api/auth/me	Dane zalogowanego użytkownika	Zalogowany
GET	/api/equipment	Lista sprzętu (z filtrami)	Zalogowany
POST	/api/equipment	Dodanie nowego sprzętu	Admin
PUT	/api/equipment/{id}	Aktualizacja sprzętu	Admin
DELETE	/api/equipment/{id}	Archiwizacja sprzętu	Admin
GET	/api/reservation	Aktywne rezerwacje użytkownika	Zalogowany
POST	/api/reservation	Utworzenie rezerwacji	Zalogowany
PUT	/api/reservation/complete/{id}	Zgłoszenie zwrotu sprzętu	Zalogowany
PUT	/api/reservation/cancel/{id}	Anulowanie rezerwacji	Zalogowany
GET	/api/reservation/pending	Lista oczekujących zwrotów	Admin
PUT	/api/reservation/confirm/{id}	Potwierdzenie zwrotu przez admina	Admin
GET	/api/statistics	Statystyki systemowe	Admin
GET	/api/location	Lista lokalizacji	Zalogowany
GET	/api/equipmenttype	Lista typów sprzętu	Zalogowany

Auth		^
POST	/api/Auth/register	🔒 ▼
POST	/api/Auth/login	🔒 ▼
GET	/api/Auth/me	🔒 ▼
POST	/api/Auth/logout	🔒 ▼
POST	/api/Auth/microsoft	🔒 ▼
Equipment		^
GET	/api/Equipment	🔒 ▼
POST	/api/Equipment	🔒 ▼
GET	/api/Equipment/{id}	🔒 ▼
PUT	/api/Equipment/{id}	🔒 ▼
DELETE	/api/Equipment/{id}	🔒 ▼
EquipmentStatus		^
GET	/api/EquipmentStatus	🔒 ▼
EquipmentType		^
GET	/api/EquipmentType	🔒 ▼
POST	/api/EquipmentType	🔒 ▼
PUT	/api/EquipmentType/{id}	🔒 ▼
DELETE	/api/EquipmentType/{id}	🔒 ▼
Location		^
GET	/api/Location	🔒 ▼
POST	/api/Location	🔒 ▼
PUT	/api/Location/{id}	🔒 ▼
DELETE	/api/Location/{id}	🔒 ▼
Reservation		^
GET	/api/Reservation	🔒 ▼
POST	/api/Reservation	🔒 ▼
GET	/api/Reservation/active/{id}	🔒 ▼
GET	/api/Reservation/{id}	🔒 ▼
GET	/api/Reservation/history	🔒 ▼
GET	/api/Reservation/history/equipment/{id}	🔒 ▼
PUT	/api/Reservation/complete/{id}	🔒 ▼
PUT	/api/Reservation/cancel/{id}	🔒 ▼
GET	/api/Reservation/dates/{id}	🔒 ▼
GET	/api/Reservation/active-by-equipment/{id}	🔒 ▼
GET	/api/Reservation/pending	🔒 ▼
GET	/api/Reservation/pending/{id}	🔒 ▼
PUT	/api/Reservation/confirm/{id}	🔒 ▼
GET	/api/Reservation/admin	🔒 ▼
Statistics		^
GET	/api/Statistics	🔒 ▼

Rysunek 6. Swagger UI z dokumentacją endpointów REST API

5.3. Baza danych

Warstwą trwałości danych jest relacyjna baza danych PostgreSQL w wersji 16. Mapowanie obiektowo-relacyjne realizowane jest przez Entity Framework Core 8 z providerem Npgsql. Schemat bazy danych zarządzany jest przez migracje EF Core – w projekcie zdefiniowano 11 migracji odzwierciedlających ewolucję schematu w trakcie procesu wytwarzania oprogramowania.

Interfejsy repozytoriów zdefiniowane są w warstwie Domain i implementowane w warstwie Infrastructure, zgodnie z zasadą odwrócenia zależności (DIP). Generyczne repozytorium `IRepository<T>` dostarcza podstawowe operacje CRUD, natomiast dedykowane repozytoria (`IReservationRepository`, `IEquipmentRepository`, `IUserRepository`) rozszerzają je o metody specyficzne dla danej encji – np. `GetConflictingReservationAsync` sprawdzający konflikt dat rezerwacji.

Przy uruchomieniu aplikacji DataSeeder wypełnia bazę danych danymi testowymi: 12 użytkownikami (2 administratorów, 10 pracowników), 29 pozycjami sprzętu z 7 typów, 6 lokalizacjami, 29 rezerwacjami (aktywne, oczekujące, zakończone, anulowane) oraz 12 defektami. Dane te umożliwiają natychmiastowe testowanie wszystkich funkcjonalności po wdrożeniu.

Tabela 3. Typy sprzętu i liczba pozycji w danych testowych

Typ sprzętu	Liczba pozycji	Przykłady
Laptop	6	Dell XPS 15, MacBook Pro 14", Lenovo ThinkPad X1 Carbon
Camera	6	Sony Alpha A7 IV, Canon EOS R5, DJI Air 3 Drone
Projector	4	Epson EB-L200F, Optoma UHD38x, BenQ MH560
Power Tool	5	Bosch GSR 18V-55, DeWalt DCS331N, Festool TSC 55
Audio Equipment	5	Rode NT-USB Mini, JBL Eon615, Shure SM58
Networking	2	Cisco Catalyst 9200L, Ubiquiti UniFi Dream Machine
Measurement	2	Fluke 87V Multimeter, Testo 875-1i Thermal Camera

5.4. Docker

Wdrożenie aplikacji opiera się na Docker Compose, który uruchamia trzy usługi: bazę danych PostgreSQL, backend .NET oraz frontend Angular. Podejście to zapewnia spójność środowiska między maszyną deweloperską a serwerem produkcyjnym oraz eliminuje problemy z zależnościami systemowymi.

Backend budowany jest wieloetapowo (multi-stage build): na etapie build używany jest obraz dotnet/sdk:8.0 do kompilacji i publikacji aplikacji, a na etapie runtime lżejszy obraz dotnet/aspnet:8.0 zawierający tylko środowisko wykonawcze. Frontend budowany jest w oparciu o node:20-alpine do kompilacji Angular, a następnie statyczne pliki serwowane są przez nginx:alpine. Konfiguracja Nginx generowana jest dynamicznie przy starcie kontenera za pomocą envsubst, co pozwala przekazać URL backendu przez zmienną środowiskową BACKEND_URL.

Usługa PostgreSQL posiada skonfigurowany healthcheck (pg_isready), a uruchomienie backendu uzależnione jest od jego pomyślnego przejścia (condition: service_healthy). Dane bazy danych przechowywane są w wolumenie Docker postgres_data, co zapewnia trwałość danych między restartami kontenera. Konfiguracja środowiskowa (hasła, klucze JWT, dane SMTP) przekazywana jest przez zmienne środowiskowe zdefiniowane w pliku .env.

5.5. Testy

Projekt GearFlow.Tests zawiera testy jednostkowe weryfikujące logikę command handlerów warstwy Application. Do pisania testów użyto frameworka xUnit, biblioteki Moq do tworzenia mocków repozytoriów i serwisów oraz biblioteki FluentAssertions do czytelnych asercji.

Testy pokrywają handlersy dla wszystkich kluczowych operacji: rejestrację i logowanie użytkownika, pobieranie danych zalogowanego użytkownika, tworzenie/aktualizację/archiwizację sprzętu, zarządzanie typami sprzętu i lokalizacjami, tworzenie, anulowanie, kończenie i potwierdzanie zwrotu rezerwacji. Każdy handler testowany jest w wielu scenariuszach: ścieżka sukcesu oraz przypadki wyjątkowe (np. sprzęt niezaleziony, przekroczony limit dni, konflikt rezerwacji, błędne hasło).

Przykładowe scenariusze testów dla CreateReservationCommandHandler obejmują: poprawne utworzenie rezerwacji z datą w przyszłości, rzucenie wyjątku KeyNotFoundException

gdy sprzęt o podanym ID nie istnieje, rzucenie `InvalidOperationException` gdy liczba dni przekracza `MaxLoanDays`, wykrycie konfliktu z istniejącą rezerwacją oraz automatyczną zmianę statusu sprzętu na `Reserved` gdy data rozpoczęcia jest dzisiaj lub wcześniej.

Tabela 4. Zestawienie testów jednostkowych według modułów

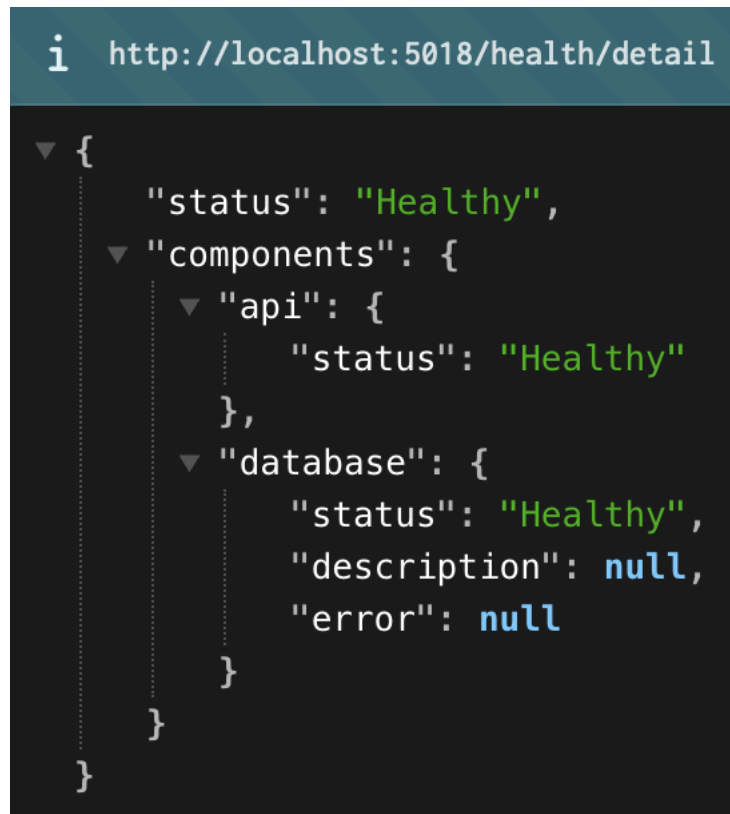
Modul	Liczba klas testowych	Przykładowe scenariusze
Auth	3	Logowanie, rejestracja, pobieranie danych użytkownika
Equipment	5	Tworzenie, aktualizacja, archiwizacja, pobieranie
EquipmentType	3	Tworzenie, aktualizacja, archiwizacja typu sprzętu
Location	3	Tworzenie, aktualizacja, archiwizacja lokalizacji
Reservation	4	Tworzenie, anulowanie, kończenie, potwierdzanie zwrotu

5.6. Monitoring i powiadomienia

System dysponuje dwoma endpointami health check: `/health` zwraca prosty tekst "Healthy" lub "Unhealthy", natomiast `/health/detail` zwraca szczegółowy JSON z informacją o stanie API i połączenia z bazą danych. Endpointy te mogą być zintegrowane z zewnętrznymi systemami monitorowania (np. Uptime Kuma, Datadog, Railway health checks) w celu automatycznego alertowania o awariach.

Powiadomienia e-mail realizowane są za pomocą biblioteki MailKit z protokołem SMTP. System wysyła wiadomości w trzech sytuacjach: potwierdzenie utworzenia rezerwacji (natychmiastowe), przypomnienie o zbliżającym się terminie zwrotu (wysyłane dzień przed upływem terminu) oraz powiadomienie po potwierdzeniu zwrotu przez administratora. Szablony wiadomości zbudowane są w formacie HTML.

Usługa `ReservationStatusUpdaterService`, działająca jako `IHostedService` w tle, uruchamia się codziennie o północy i automatycznie aktualizuje stany sprzętu: sprzęt powiązany z aktywną rezerwacją, której data rozpoczęcia minęła, otrzymuje status `Reserved` (2), sygnalizując że powinien być aktualnie u wypożyczającego. Usługa ta eliminuje konieczność ręcznej aktualizacji statusów przez administratora.



```
i http://localhost:5018/health/detail
{
  "status": "Healthy",
  "components": {
    "api": {
      "status": "Healthy"
    },
    "database": {
      "status": "Healthy",
      "description": null,
      "error": null
    }
  }
}
```

Rysunek 7. Odpowiedź endpointu /health/detail

6. Podsumowanie i wnioski

W ramach projektu GearFlow zrealizowano kompletną, pełnostackową aplikację webową do zarządzania rezerwacjami sprzętu. System składa się z backendowego REST API zbudowanego w ASP.NET Core 8 z zastosowaniem Clean Architecture i wzorca CQRS, frontendowej aplikacji SPA w Angular 21 z biblioteką PrimeNG oraz bazy danych PostgreSQL. Całość wdrażana jest przy użyciu Docker Compose.

Projekt pozwolił na praktyczne zastosowanie wielu wzorców projektowych i dobrych praktyk inżynierii oprogramowania: Clean Architecture zapewniła niezależność warstwy domenowej od infrastruktury, wzorec CQRS wymusił wyraźny podział między operacjami odczytu i zapisu, wzorec Repository pozwolił odizolować logikę biznesową od szczegółów dostępu do danych, a middleware pipeline w ASP.NET Core umożliwił centralne zarządzanie cross-cutting concerns takich jak obsługa błędów i uwierzytelnianie.

W trakcie realizacji projektu napotkano następujące wyzwania: konfiguracja Microsoft SSO z Azure Active Directory wymagała dokładnego zrozumienia przepływu OAuth2/OIDC i

właściwego parsowania tokenów MSAL po stronie serwera; zarządzanie cyklem życia rezerwacji (szczególnie logika przejść między stanami Active, Pending i Completed) wymagała precyzyjnego modelowania przepływu po stronie użytkownika i administratora; konfiguracja CORS dla środowisk deweloperskiego i produkcyjnego wymagała odrębnego traktowania.

Możliwe kierunki dalszego rozwoju aplikacji obejmują: wdrożenie powiadomień push przez WebSockets (SignalR) zamiast wyłącznie e-mail, rozbudowę modułu defektów o pełny system zgłoszeń serwisowych z przepływem naprawy, dodanie widoku kalendarza z graficzną prezentacją dostępności sprzętu, implementację skanowania kodów QR dla sprzętu przy pomocy aplikacji mobilnej, oraz wdrożenie pipeline CI/CD z automatycznym testowaniem i wdrożeniem na platformę Railway lub podobną.

Spis tabel

Tabela 1. Główne komponenty architektury systemu GearFlow

Tabela 2. Główne endpointy REST API systemu GearFlow

Tabela 3. Typy sprzętu i liczba pozycji w danych testowych

Tabela 4. Zestawienie testów jednostkowych według modułów

Spis rysunków

Rysunek 1. Przykładowy interfejs jednego z analizowanych systemów

Rysunek 2. Diagram architektury warstwowej systemu GearFlow

Rysunek 3. Diagram encji i relacji (ERD) bazy danych GearFlow

Rysunek 4. Widok listy sprzętu z filtrowaniem

Rysunek 5. Panel statystyk administratora

Rysunek 6. Swagger UI z dokumentacją endpointów REST API

Rysunek 7. Odpowiedź endpointu /health/detail

Bibliografia

- [1] Microsoft, ASP.NET Core documentation, docs.microsoft.com, 2024.
- [2] Google, Angular - The modern web developer's platform, angular.dev, 2024.
- [3] PrimeNG Team, PrimeNG - Angular UI Component Library, primeng.org, 2024.
- [4] Martin, R.C., Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2017.
- [5] Newman, S., Building Microservices: Designing Fine-Grained Systems, O'Reilly Media, 2021.
- [6] Entity Framework Core Team, EF Core documentation, learn.microsoft.com, 2024.
- [7] Docker Inc., Docker documentation, docs.docker.com, 2024.
- [8] PostgreSQL Global Development Group, PostgreSQL 16 Documentation, postgresql.org, 2023.